

MedSphere – Complete Semester 3 Submission Pack

SYNOPSIS

MedSphere – Unified Hospital Information Management System

MCA Semester 3 Mini Project Synopsis

Introduction

MedSphere is a Semester 3 MCA mini project providing a centralized web-based system for core hospital information and workflows. The current version covers patients, departments, doctors, appointments, consultations, prescriptions and medical history.

Problem Statement

Connected hospital workflows require consistent handling of patient, doctor, appointment and clinical information. MedSphere provides an academic centralized system for these core workflows through a role-based database-backed web application.

Objectives

- Centralize core hospital information.
- Provide role-based access for ADMIN, RECEPTIONIST and DOCTOR.
- Manage patients, departments and doctors.
- Schedule and manage appointments.
- Record consultations and prescriptions.
- Maintain patient medical history.
- Apply validation and business rules.
- Demonstrate secure authentication and authorization.

Technology Stack

Java 21; Spring Boot 3.5.5; Spring MVC; Spring Security; Spring Data JPA; Hibernate; Thymeleaf; Bootstrap 5.3.3; JavaScript; MariaDB/MySQL; Maven Wrapper; XAMPP.

Architecture

Browser/UI → Controller → Service → Repository → JPA/Hibernate → MariaDB/MySQL.

SDLC

Planning → Requirement Analysis → System Design → Implementation → Testing → Deployment/Operation → Maintenance/Future Enhancement.

Expected Outcome

A working academic hospital information management application demonstrating secure role-based access and end-to-end database-backed workflows.

Conclusion

MedSphere demonstrates how Java and Spring Boot technologies can be combined to implement a structured hospital information management application within Semester 3 scope.

PROJECT REPORT

MedSphere – Unified Hospital Information Management System

MCA Semester 3 Final Project Report

1. Introduction

MedSphere is a Java/Spring Boot hospital information management application developed as an MCA Semester 3 mini project.

2. Problem Statement

Hospital information workflows connect patients, doctors, departments, appointments and clinical records. MedSphere provides a centralized academic system for these core workflows.

3. Objectives

Centralized patient information; role-based access; department and doctor management; appointment scheduling; consultation and prescription recording; medical history; validation; persistence; secure authentication.

4. Functional Modules

Authentication; Patient Management; Department Management; Doctor Management; Appointment Management; Doctor Consultation; Prescription Management; Medical History; Dashboard and responsive UI.

5. Technology

Java 21; Spring Boot 3.5.5; Spring MVC; Spring Security; Spring Data JPA; Hibernate; Thymeleaf; Bootstrap 5.3.3; JavaScript; MariaDB/MySQL; Maven Wrapper; XAMPP.

6. Architecture

Browser/UI → Controller → Service → Repository → JPA/Hibernate → MariaDB/MySQL.

7. Security

Database-backed authentication, BCrypt password hashing, CSRF protection and role-based authorization.

8. Database

Main entities are Role, User, Department, Doctor, Patient, Appointment, Consultation, Prescription and MedicalHistory. Patient codes use MSP-YYYY-000001.

9. Core Workflows

Patient registration → patient code → profile/history.

Appointment creation → conflict validation → SCHEDULED.

Doctor consultation → save → COMPLETED.

Consultation → prescription.

Admin → doctor account and profile.

10. Testing

Manual functional testing covered authentication, authorization, CRUD workflows, validation, appointment conflicts, consultation/prescription flow and persistence after restart.

11. Limitations

Billing, pharmacy inventory, laboratory management, wards/beds, advanced notifications and analytics are not implemented in the Semester 3 version.

12. Future Scope

Semester 4 can expand EMR, laboratory/radiology, pharmacy, billing, admission/discharge, bed management, notifications, analytics, backup/recovery and multi-building/multi-branch architecture.

13. Conclusion

MedSphere demonstrates an end-to-end Java/Spring Boot hospital information workflow with secure role-based access, relational persistence and layered architecture.

PROJECT DIARY / WORKBOOK

MedSphere – Project Diary / Workbook

Academic Progress Record

Important

Use these as progress headings and enter the actual dates/signatures required by the college. Do not invent dates or signatures.

Progress

1. Project domain/problem identified
2. Scope/objectives finalized
3. Requirements identified
4. Roles defined
5. Architecture designed
6. Database designed
7. Spring Boot structure prepared
8. Authentication/security implemented
9. Patient management implemented
10. Department management implemented
11. Doctor management implemented
12. Appointment module implemented
13. Consultation implemented
14. Prescription implemented
15. Medical history implemented
16. UI refined
17. Validation/testing
18. Database persistence verified
19. Documentation prepared
20. Diagrams prepared
21. Presentation prepared
22. Final submission preparation

USER MANUAL

MedSphere – Unified Hospital Information Management System

User Manual

1. Prerequisites

Java 21, XAMPP MariaDB/MySQL, browser, Git and project source.

2. Database

Start XAMPP MariaDB/MySQL on port 3307. Keep XAMPP Tomcat off because Spring Boot uses port 8080.

3. Application

Run `.mvnw.cmd` `spring-boot:run` and open `http://localhost:8080/login`.

4. Development Accounts

admin / admin123 — ADMIN

receptionist / reception123 — RECEPTIONIST

doctor / doctor123 — DOCTOR

These are development/demo credentials only.

5. Admin

Login → Admin Dashboard → departments → doctors.

6. Receptionist

Login → Reception → patient registration/search/profile → appointment → medical history.

7. Doctor

Login → My Appointments → assigned appointment → consultation → prescription.

8. Security

Spring Security enforces role-specific protected URLs.

9. Troubleshooting

Check MariaDB port 3307, application port 8080, Java 21 and Maven Wrapper.

10. Scope

Semester 3 does not include billing, pharmacy inventory, laboratory management or wards/beds.

TEAM RESPONSIBILITIES

MedSphere – 3-Student Module Responsibility Plan

Viva / Presentation Organization

Student 1 – Administration & Security

Authentication; Spring Security; roles; BCrypt/CSRF; Admin dashboard; departments; doctor management/account creation.

Student 2 – Patient & Reception

Patient registration; patient-code generation; patient search/profile/edit; medical history; appointment workflow; Reception dashboard.

Student 3 – Doctor & Clinical Workflow

Doctor dashboard; assigned appointments; consultation; prescription; consultation details; clinical UI.

Shared Topics

Objective/problem; architecture; JPA/Hibernate; database; SDLC; testing; limitations; future scope.

Truthfulness Rule

This allocation is for viva/presentation organization. Do not falsely claim authorship of code.

VIVA PREPARATION

MedSphere – Student-wise Viva Preparation

MCA Semester 3

Student 1

Introduction; authentication; Spring Security; roles; BCrypt; CSRF; Admin; departments; doctor account creation.

Student 2

Patient registration/code; profile/search/edit; medical history; appointment creation; conflict validation; status.

Student 3

Doctor dashboard; assigned appointments; consultation; prescription; ownership checks; completion workflow; UI.

Every Student

Objective; problem; technology; MVC architecture; JPA/Hibernate; relationships; database; validation; testing; limitations; future scope.

Answer Pattern

What → Why → How → Example.

Truthfulness

Do not claim to have personally written code that you did not write.

FINAL DEMO SCRIPT

MedSphere – Final Demo Script

End-to-End Demonstration

Admin Phase

Login → Admin Dashboard → Departments → Create/show Doctor.

Reception Phase

Login as Receptionist → Patient → Show patient code → Create Appointment → show SCHEDULED.

Doctor Phase

Login as Doctor → My Appointments → Consultation → Save → show COMPLETED → Add Prescription.

Patient Record

Show Medical History and explain the connected workflow.

Viva Points

Controller → Service → Repository → Database; Spring Security; BCrypt; CSRF; conflict validation; consultation completion; prescription relationship.

Demo Rule

Do not claim a feature is implemented if it is not present in the current build.